

Sheet 9

Monday 8th December, 2025

Please hand in a zip-archive, containing a pdf for exercise 1, 2 and 3, as well as the plots for exercise 2 and 3, and the movie for exercise 2, which is named `<firstname>_<lastname>.sheet_<number>`.

The deadline is Monday, Dec. 15th, 12:00 (noon). Please hand in your solution via moodle.

1 Euler and Verlet algorithms

In previous sheets we already made use of numerical methods to integrate ordinary differential equations. Now that we've arrived at molecular dynamics (MD), these are more relevant than ever. The problem that needs to be solved is always the same: We have Newton's equations of motion

$$\frac{\partial^2 \mathbf{R}_I(t)}{\partial t^2} = -\frac{1}{M_I} \nabla_I V_{BO}(\mathbf{R}(t)) = \mathbf{a}_I(\mathbf{R}(t)), \quad (1)$$

where $\mathbf{R}_I(t)$ is the position of ion I with mass M_I at time t , $\mathbf{R}(t)$ refers to the set of positions of all ions, and $\mathbf{a}_I$ is the acceleration of ion I (∇_I is the gradient with respect to the coordinate $\mathbf{R}_I$). We calculate V_{BO} e.g. using DFT, and can therefore assume that the acceleration for any set of coordinates is known.

Solving this equation analytically is impossible, so we turn to numerical methods. We start for some configuration of positions and velocities $\{\mathbf{R}(t=0); \dot{\mathbf{R}}(t=0)\} = \{\mathbf{R}_0; \dot{\mathbf{R}}_0\}$, and perform the time evolution in a series of discrete steps. Depending on how sophisticated our algorithm for performing the time evolution is, the result will be more or less close to the true trajectory.

1. Derive the Euler algorithm

$$\mathbf{R}(t + \Delta t) = \mathbf{R}(t) + \Delta t \cdot \dot{\mathbf{R}}(t) + \mathcal{O}(\Delta t^2) \quad (2)$$

$$\dot{\mathbf{R}}(t + \Delta t) = \dot{\mathbf{R}}(t) + \Delta t \cdot \mathbf{a}(\mathbf{R}(t)) + \mathcal{O}(\Delta t^2) \quad (3)$$

by writing out the Taylor expansions around time t for the positions and velocities and truncating after the linear terms.

1 point

2. Show that the Euler-algorithm is in general not time-reversible by performing one step in the forward ($+\Delta t$) and one step in the backward ($-\Delta t$) direction, and showing

that the result is not necessarily equal to the initial conditions.

Hint: To go back one step, you can use the same equations while replacing $\Delta t \rightarrow -\Delta t$ in all expressions.

1 point

3. Derive the Verlet-algorithm

$$\mathbf{R}(t + \Delta t) = 2\mathbf{R}(t) - \mathbf{R}(t - \Delta t) + (\Delta t)^2 \mathbf{a}(\mathbf{R}(t)) + \mathcal{O}(\Delta t^4) \quad (4)$$

by writing out the Taylor expansions around the position for $(+\Delta t)$ and $(-\Delta t)$, and summing them together. Show that this algorithm **does** fulfil time reversal symmetry by showing that performing a single step forwards and a single step backwards results in the initial conditions.

Hint: To get an expression for $\mathbf{a}(\mathbf{R}(t + \Delta t))$, calculate $\mathbf{R}(t + \Delta t + \Delta t)$.

1 point

Once this is shown, it is possible to prove that time-reversal symmetry (TRS) is true for all times by induction. Assume that the algorithm fulfils TRS for N steps. Add one step. Since we have shown that TRS is given for one step, we use the configuration at step N as the initial conditions for step $N + 1$. We can reverse from step $N + 1$ back to step N , and then by assumption back to step 1. Therefore TRS is given for any finite number of steps N .

Force-field MD simulations

The following two exercises will introduce you to *molecular dynamics* (MD) simulations, and especially the open-source MD package *LAMMPS*. LAMMPS is already installed in the virtual machine. A documentation, including information on how to install it on your personal computer, can be found here: <https://lammps.sandia.gov/doc/Manual.html>. On Ubuntu you can alternatively install it from the repositories by executing `sudo apt install lammps`.

2 MD simulation of Si (NVE)

In moodle, you find three input files for a Si calculation: the *Si.atoms* containing information about the simulation cell, *Si.in* containing all input parameters for the calculation, and the file *SiCGe.tersoff* containing information on the force-field.

1. Go through the input file *Si.in* and give a short explanation of each parameter used.
2. Change the input such that 10 ps will be simulated with a 1 fs timestep.
3. Adapt the line starting with *thermo_style* such that the following observables are written out:

- step
- volume
- pressure
- temperature
- potential energy
- kinetic energy
- total energy

4. Perform an NVE calculation

```
lammps < Si.in > Si.out
```

5. Plot volume, pressure, temperature, and total energies as functions of time. Which observables are conserved? Which observables did you expect to be conserved?
Please use a different plot for each quantity and provide units.
6. Calculate the average values and standard deviation of the above observables.
7. Plot the temperature as a function of the kinetic energy and explain the result. Why do we have a non-zero temperature?
8. Create a movie of the MD trajectory.
Use, e.g., *dump m1 all image 100 pic*.jpg type type* in the .in file for a lammps calculation to create a large number of individual images (frames) and combine them into a movie afterwards. One method to combine them to a movie using the virtual machine is to run the commands

```
sudo apt install ffmpeg
```

```
sudo apt install imagemagick-6.q16hdri
```

```
convert -loop 1 *.jpeg movie.mpg
```

inside the folder with the images.

4 points

3 NVE simulation starting from the relaxed equilibrium structure with an initial velocity distribution

1. Perform a calculation finding the equilibrium structure of Si using lammps.
Hints: look for the parameters fix box/relax and minimize in the documentation.
2. What are the equilibrium volume and total energy of the system?
3. Provide the atoms with a random initial velocity drawn from a distribution at 300K.
Hint: use velocity all create
4. Perform an NVE calculation. What do you observe for the temperature?

3 points